

9 *Functions*

This chapter covers

- Defining functions
- Using function parameters
- Passing mutable objects as parameters
- Understanding local and global variables
- Creating and using generator functions
- Creating and using lambda expressions
- Using decorators

This chapter assumes that you're familiar with function definitions in at least one other computer language and with the concepts that correspond to function definitions, arguments, parameters, and so forth.

9.1 Basic function definitions

The basic syntax for a Python function definition is

```
def name(parameter1, parameter2, . . .):  
    body
```

As it does with control structures, Python uses indentation to delimit the body of the function definition. The following simple example puts the factorial code from a previous section into a function body, so you can call a `fact` function to obtain the factorial of a number:

```
>>> def fact(n):
...     """Return the factorial of the given number."""
...     r = 1
...     while n > 0:
...         r = r * n
...         n = n - 1
...     return r
... 
```

← 1

← 2

The second line 1 is an optional *documentation string*, or *docstring*. You can obtain its value by printing `fact.__doc__`. The intention of docstrings is to describe the external behavior of a function and the parameters it takes, whereas comments should document internal information about how the code works. Docstrings are strings that immediately follow the first line of a function definition and are usually triple quoted to allow for multiline descriptions. Browsing tools are available that extract the first line of document strings. It's standard practice for multiline documentation strings to give a synopsis of the function in the first line, follow this synopsis with a blank second line, and end with the rest of the information. This line indicates that the value after the return is sent back to the code calling the function 2.

Procedure or function?

In some languages, a function that doesn't return a value is called a *procedure*. Although you can (and will) write functions that don't have a `return` statement, they aren't really procedures. All Python procedures are functions; if no explicit `return` is executed in the procedure body, the special Python value `None` is returned, and if `return arg` is executed, the value `arg` is immediately returned. Nothing else in the function body is executed after a `return` has been executed. Because Python doesn't have true procedures, I'll refer to both types as *functions*.

Although all Python functions return values, it's up to you whether a function's return value is used:

```
>>> fact(4)
24
>>> x = fact(4)
>>> x
24
>>> 
```

← 1

← 2

← 3

The return value isn't associated with a variable 1. The `fact` function's value is printed in the interpreter only 2. The return value is associated with the variable `x` 3.

9.2 Function parameter options

Most functions need parameters, and each language has its own specifications for how function parameters are defined. Python is flexible and provides three options for defining function parameters. These options are outlined in this section.

9.2.1 Positional parameters

The simplest way to pass parameters to a function in Python is by position. In the first line of the function, you specify variable names for each parameter; when the function is called, the parameters used in the calling code are matched to the function's parameter variables based on their order. The following function computes x to the power of y :

```
>>> def power(x, y):
...     r = 1
...     while y > 0:
...         r = r * x
...         y = y - 1
...     return r
...
>>> power(3, 3)
27
```

This method requires that the number of parameters used by the calling code exactly matches the number of parameters in the function definition; otherwise, a `TypeError` exception is raised:

```
>>> power(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: power() missing 1 required positional argument: 'y'
>>>
```

Default values

Function parameters can have default values, which you declare by assigning a default value in the first line of the function definition, like so:

```
def fun(arg1, arg2=default2, arg3=default3, . . .)
```

Any number of parameters can be given default values. Parameters with default values must be defined as the last ones in the parameter list because Python, like most languages, pairs arguments with parameters on a positional basis. There must be enough arguments to a function that the last parameter in that function's parameter list without a default value gets an argument. See section 9.2.2, "Passing arguments by parameter name," for a more flexible mechanism.

The following function also computes x to the power of y . But if y isn't given in a call to the function, the default value of 2 is used, and the function is just the square function:

```
>>> def power(x, y=2):
...     r = 1
...     while y > 0:
...         r = r * x
...         y = y - 1
...     return r
...
```

You can see the effect of the default argument in the following interactive session:

```
>>> power(3, 3)
27
>>> power(3)
9
```

9.2.2 Passing arguments by parameter name

You can also pass arguments into a function by using the name of the corresponding function parameter rather than its position. Continuing with the previous interactive example, you can type

```
>>> power(2, 3)
8
>>> power(3, 2)
9
>>> power(y=2, x=3)
9
```

Because the arguments to `power` in the final invocation are named, their order is irrelevant; the arguments are associated with the parameters of the same name in the definition of `power`, and you get back 3^2 . This type of argument passing is called *keyword passing*.

Keyword passing, in combination with the default argument capability of Python functions, can be highly useful when you're defining functions with large numbers of possible arguments, most of which have common defaults. Consider a function that's intended to produce a list with information about files in the current directory and that uses Boolean arguments to indicate whether that list should include information such as file size, last modified date, and so forth, for each file. You can define such a function along these lines

```
def list_file_info(size=False, create_date=False, mod_date=False, ...):
    ...get file names...
    if size:
        # code to get file sizes goes here
    if create_date:
        # code to get create dates goes here
    # do any other stuff desired

    return fileinfostructure
```

and then call it from other code using keyword argument passing to indicate that you want only certain information (in this example, the file size and modification date but *not* the creation date):

```
fileinfo = list_file_info(size=True, mod_date=True)
```

This type of argument handling is particularly suited for functions with very complex behavior, and one place where such functions occur is in a graphical user interface (GUI). If you ever use the Tkinter package to build GUIs in Python, you'll find that the use of optional, keyword-named arguments like this is invaluable.

9.2.3 Variable numbers of arguments

Python functions can also be defined to handle variable numbers of arguments, which you can do in two ways. One way handles the relatively familiar case in which you want to collect an unknown number of arguments at the end of the argument list into a list. The other method can collect an arbitrary number of keyword-passed arguments, which have no correspondingly named parameter in the function parameter list, into a dictionary. These two mechanisms are discussed next.

DEALING WITH AN INDEFINITE NUMBER OF POSITIONAL ARGUMENTS

Prefixing the final parameter name of the function with a `*` causes all excess non-keyword arguments in a call of a function (that is, those positional arguments not assigned to another parameter) to be collected together and assigned as a tuple to the given parameter. Here's a simple way to implement a function to find the maximum in a list of numbers.

First, implement the function:

```
>>> def maximum(*numbers):
...     if len(numbers) == 0:
...         return None
...     else:
...         maxnum = numbers[0]
...         for n in numbers[1:]:
...             if n > maxnum:
...                 maxnum = n
...         return maxnum
... 
```

Now test the behavior of the function:

```
>>> maximum(3, 2, 8)
8
>>> maximum(1, 5, 9, -2, 2)
9
```

DEALING WITH AN INDEFINITE NUMBER OF ARGUMENTS PASSED BY KEYWORD

An arbitrary number of keyword arguments can also be handled. If the final parameter in the parameter list is prefixed with `**`, it collects all excess *keyword-passed* arguments into a dictionary. The key for each entry in the dictionary is the keyword

(parameter name) for the excess argument. The value of that entry is the argument itself. An argument passed by keyword is excess in this context if the keyword by which it was passed doesn't match one of the parameter names in the function definition.

For example:

```
>>> def example_fun(x, y, **other):
...     print("x: {0}, y: {1}, keys in 'other': {2}".format(x,
...         y, list(other.keys())))
...     other_total = 0
...     for k in other.keys():
...         other_total = other_total + other[k]
...     print("The total of values in 'other' is {0}".format(other_total))
```

Trying out this function in an interactive session reveals that it can handle arguments passed in under the keywords `foo` and `bar`, even though `foo` and `bar` aren't parameter names in the function definition:

```
>>> example_fun(2, y="1", foo=3, bar=4)
x: 2, y: 1, keys in 'other': ['foo', 'bar']
The total of values in 'other' is 7
```

9.2.4 *Mixing argument-passing techniques*

It's possible to use all of the argument-passing features of Python functions at the same time, although it can be confusing if not done with care. The general rule for using mixed argument-passing is that positional arguments come first, then named arguments, followed by the indefinite positional argument with a single `*`, and last of all the indefinite keyword argument with `**`. See the documentation for full details.

QUICK CHECK: FUNCTIONS AND PARAMETERS How would you write a function that could take any number of unnamed arguments and print their values out in reverse order?

What do you need to do to create a procedure or void function—that is, a function with no return value?

What happens if you capture the return value of a function with a variable?

9.3 *Mutable objects as arguments*

Arguments are passed in by object reference. The parameter becomes a new reference to the object. For immutable objects (such as tuples, strings, and numbers), what is done with a parameter has no effect outside the function. But if you pass in a mutable object (such as a list, dictionary, or class instance), any change made to the object changes what the argument is referencing outside the function. Reassigning the parameter doesn't affect the argument, as shown in figures 9.1 and 9.2:

```
>>> def f(n, list1, list2):
...     list1.append(3)
...     list2 = [4, 5, 6]
...     n = n + 1
...
>>> x = 5
>>> y = [1, 2]
```

```
>>> z = [4, 5]
>>> f(x, y, z)
>>> x, y, z
(5, [1, 2, 3], [4, 5])
```

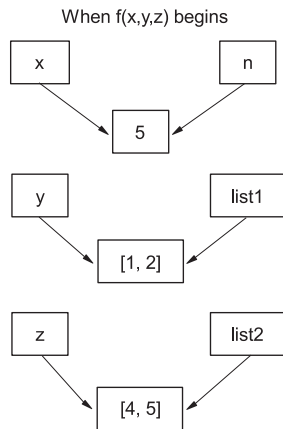


Figure 9.1 At the beginning of function $f()$, both the initial variables and the function parameters refer to the same objects.

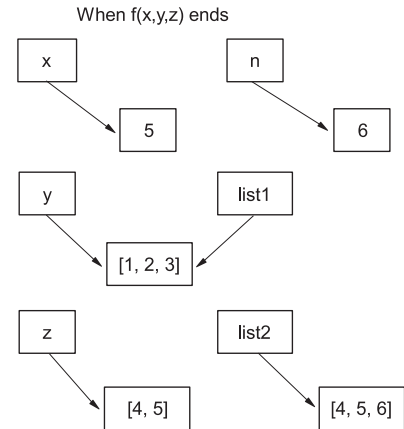


Figure 9.2 At the end of function $f()$, y ($list1$ inside the function) has been changed internally, whereas n and $list2$ refer to different objects.

Figures 9.1 and 9.2 illustrate what happens when function f is called. The variable x isn't changed because it's immutable. Instead, the function parameter n is set to refer to the new value of 6. Likewise, variable z is unchanged because inside function f , its corresponding parameter $list2$ was set to refer to a new object, $[4, 5, 6]$. Only y sees a change because the actual list it points to was changed.

QUICK CHECK: MUTABLE FUNCTION PARAMETERS What would be the result of changing a list or dictionary that was passed into a function as a parameter value? Which operations would be likely to create changes that would be visible outside the function? What steps might you take to minimize that risk?

9.4 Local, nonlocal, and global variables

Here, you return to the definition of `fact` from the beginning of this chapter:

```
def fact(n):
    """Return the factorial of the given number."""
    r = 1
    while n > 0:
        r = r * n
        n = n - 1
    return r
```

Both the variables r and n are *local* to any particular call of the factorial function; changes to them made when the function is executing have no effect on any variables

outside the function. Any variables in the parameter list of a function, and any variables created within a function by an assignment (like `r = 1` in `fact`), are local to the function.

You can explicitly make a variable global by declaring it so before the variable is used, using the `global` statement. Global variables can be accessed and changed by the function. They exist outside the function and can also be accessed and changed by other functions that declare them global or by code that's not within a function. Here's an example that shows the difference between local and global variables:

```
>>> def fun():
...     global a
...     a = 1
...     b = 2
... 
```

This example defines a function that treats `a` as a global variable and `b` as a local variable, and attempts to modify both `a` and `b`.

Now test this function:

```
>>> a = "one"
>>> b = "two"

>>> fun()
>>> a
1
>>> b
'two'
```

The assignment to `a` within `fun` is an assignment to the global variable `a` also existing outside `fun`. Because `a` is designated `global` in `fun`, the assignment modifies that global variable to hold the value `1` instead of the value `"one"`. The same isn't true for `b`; the local variable called `b` inside `fun` starts out referring to the same value as the variable `b` outside `fun`, but the assignment causes `b` to point to a new value that's local to the function `fun`.

Similar to the `global` statement is the `nonlocal` statement, which causes an identifier to refer to a previously bound variable in the closest enclosing scope. I discuss scopes and namespaces in more detail in chapter 10, but the point is that `global` is used for a top-level variable, whereas `nonlocal` can refer to any variable in an enclosing scope, as the example in listing 9.1 illustrates.

Listing 9.1 File `nonlocal.py`

```
g_var = 0
nl_var = 0
print("top level-> g_var: {0} nl_var: {1}".format(g_var, nl_var))
def test():
    nl_var = 2
    print("in test-> g_var: {0} nl_var: {1}".format(g_var, nl_var))
    def inner_test():
```

`g_var` in `inner_test`
binds top-level `g_var`.

`nl_var` in
`inner_test`
binds to
`nl_var` in `test`.


```

global g_var
nonlocal nl_var
g_var = 1
nl_var = 4
print("in inner_test-> g_var: {0} nl_var: {1}".format(g_var,
                                                    nl_var))

inner_test()
print("in test-> g_var: {0} nl_var: {1}".format(g_var, nl_var))

test()
print("top level-> g_var: {0} nl_var: {1}".format(g_var, nl_var))

```

When run, this code prints the following:

```

top level-> g_var: 0 nl_var: 0
in test-> g_var: 0 nl_var: 2
in inner_test-> g_var: 1 nl_var: 4
in test-> g_var: 1 nl_var: 4
top level-> g_var: 1 nl_var: 0

```

Note that the value of the top-level `nl_var` hasn't been affected, which would happen if `inner_test` contained the line `global nl_var`.

The bottom line is that if you want to assign to a variable existing outside a function, you must explicitly declare that variable to be nonlocal or global. But if you're accessing a variable that exists outside the function, you don't need to declare it nonlocal or global. If Python can't find a variable name in the local function scope, it attempts to look up the name in the global scope. Hence, accesses to global variables are automatically sent through to the correct global variable. Personally, I don't recommend using this shortcut. It's much clearer to a reader if all global variables are explicitly declared as global. Further, you probably want to limit the use of global variables within functions to rare occasions.

TRY THIS: GLOBAL VS. LOCAL VARIABLES Assuming that `x = 5`, what will be the value of `x` after `funct_1()` below executes? After `funct_2()` executes?

```

def funct_1():
    x = 3
def funct_2():
    global x
    x = 2

```

9.5 Assigning functions to variables

Functions can be assigned, like other Python objects, to variables, as shown in this example:

```

>>> def f_to_kelvin(degrees_f):
...     return 273.15 + (degrees_f - 32) * 5 / 9
...
>>> def c_to_kelvin(degrees_c):
...     return 273.15 + degrees_c

```

```

...
>>> abs_temperature = f_to_kelvin
>>> abs_temperature(32)
273.15
>>> abs_temperature = c_to_kelvin
>>> abs_temperature(0)
273.15

```

← Assigns function to variable

← Assigns function to variable

You can place functions in lists, tuples, or dictionaries:

```

>>> t = {'FtoK': f_to_kelvin, 'CtoK': c_to_kelvin}
>>> t['FtoK'](32)
273.15
>>> t['CtoK'](0)
273.15

```

← ①

← Accesses the `f_to_kelvin` function as value in dictionary

← Accesses the `c_to_kelvin` function as value in dictionary

A variable that refers to a function can be used in exactly the same way as the function ①. This last example shows how you can use a dictionary to call different functions by the value of the strings used as keys. This pattern is common in situations in which different functions need to be selected based on a string value, and in many cases, it takes the place of the `switch` structure found in languages such as C and Java.

9.6 *lambda expressions*

Short functions like those you just saw can also be defined by using `lambda` expressions of the form

```
lambda parameter1, parameter2, . . . : expression
```

`lambda` expressions are anonymous little functions that you can quickly define inline. Often, a small function needs to be passed to another function, like the `key` function used by a list's `sort` method. In such cases, a large function is usually unnecessary, and it would be awkward to have to define the function in a separate place from where it's used. The dictionary in the previous subsection can be defined all in one place with

```

>>> t2 = {'FtoK': lambda deg_f: 273.15 + (deg_f - 32) * 5 / 9,
...      'CtoK': lambda deg_c: 273.15 + deg_c}
>>> t2['FtoK'](32)
273.15

```

← ①

This example defines `lambda` expressions as values of the dictionary ①. Note that `lambda` expressions don't have a `return` statement because the value of the expression is automatically returned.